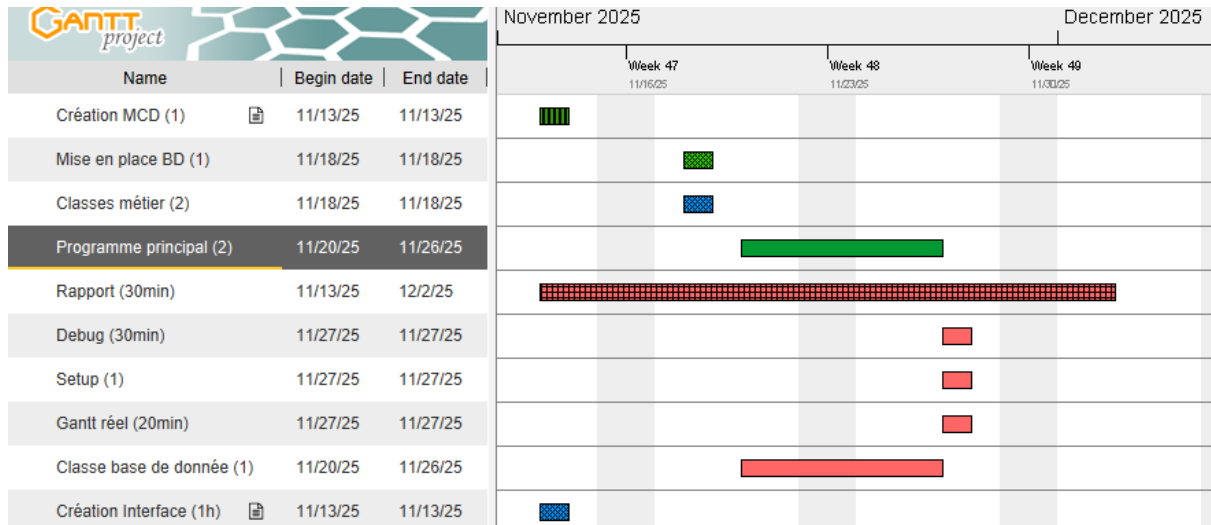


PROJET C#

13/11/2025

1. Création Gantt prévisionnel



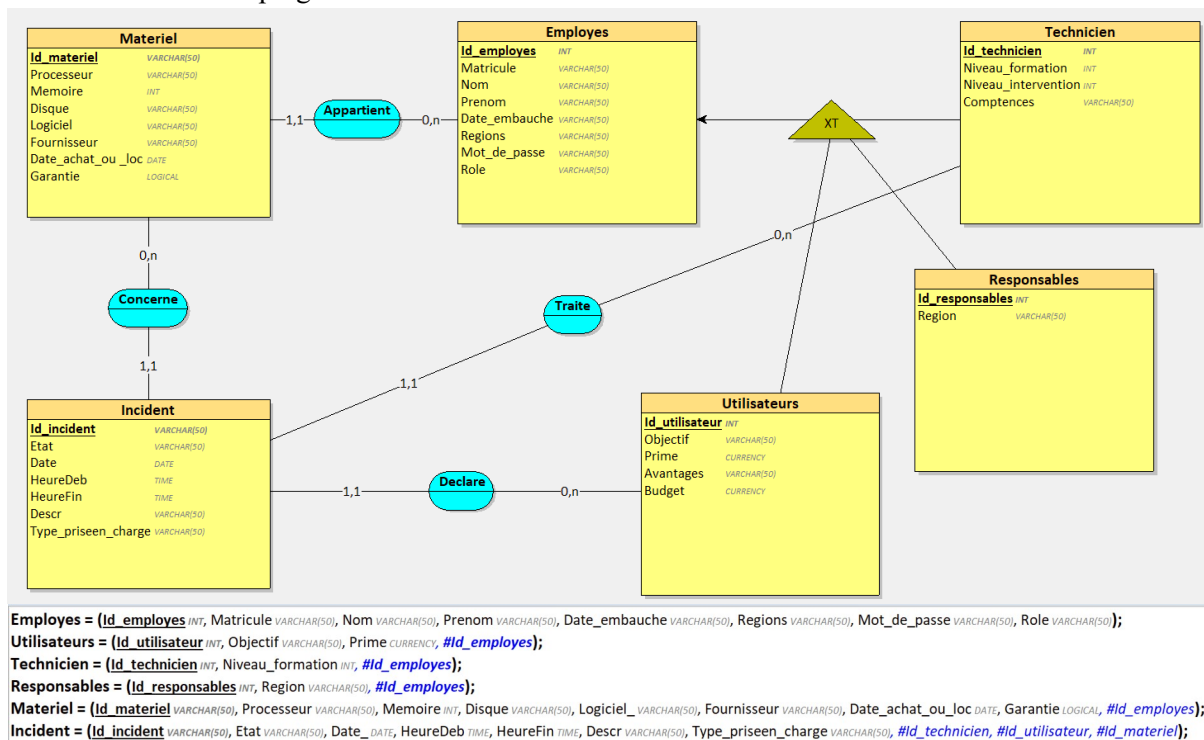
Bleu : Sanaa

Vert : Adrien

Rose : Travail partagé

(tout est évidemment partagé)

2. MCD sur looping + MLD



3. Interface

Tconnexion | Tutilisateur | TajMat | TchargeInc | TcreaEmp | Tstat

Identifiant:

Mot de passe:

T connexion	T utilisateur	TajMat	TchargeInc	TcreaEmp	Tstat
Avancement de l'incident Poste ComboBoxPoste ▾ Description TextBoxDesc Déclarer incident Suivre l'avancement ListBoxAvancement					

T connexion	T utilisateur	TajMat	TchargeInc	TcreaEmp	Tstat
Propriétaire du poste ComboBoxPropri ▾ Memoire TextboxMemoire Processeur du poste TextboxProcesseur Disque TextboxDisque Logiciels du poste TextboxLogiciels Date d'acquisition (YYYY-MM-DD) Garantie ☐ Oui ☐ Non Matériel Acheté ou loué ☐ Acheté ☐ Loué Ajouter Materiel					

Tconnexion	Tutilisateur	TajMat	TchargeInc	TcreaEmp	Tstat
Materiel comboBoxMateriel ▼ Supprimer Materiel Infos Materiel ListBoxInfosMateriel					
Prise en charge comboBoxIncider ▼ comboBoxTechni ▼ Prendre en charge Incidents non pris en charge listBoxIncidentsNonPrisEr					
Resolution comboBoxTechni ▼ Resoudre l'incident Description du travail					

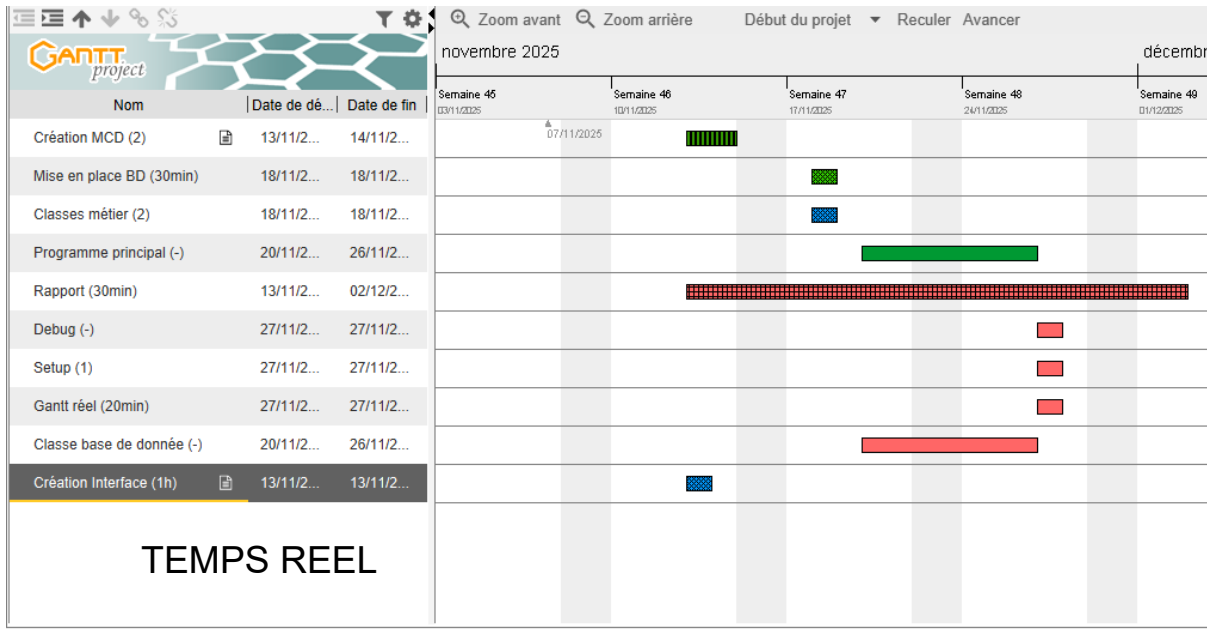
Tconnexion	Tutilisateur	Tajtmat	TchargeInc	TcreaEmp	Tstat
Créer un nouvel employé			Modifier un employé		
Matricule	textBoxMatricule		comboBoxModifier	Supprimer employés	
Nom	textBoxNom		Matricule	textBoxModifierMatric	
Prenom	textBoxPrenom		Nom	textBoxModifierNom	
Region	textBoxRegion		Prenom	textBoxModifierPreno	
Date d'embauche	mardi 9 décembre 2025		Region	textBoxModifierRegic	
Mot de passe	txtBoxMDPE		Rôle	Niveau d'études (pour les techiciens)	
Rôle	Niveau d'études (pour les techiciens)		☐ Technicien	☐ Master	
☐ Utilisateur	☐ Master		☐ Utilisateur	☐ BTS	
	☐ Baccalureat			☐ Baccalureat	
Objectif (pour les utilisateurs)	textBoxObjectifs		Objectif (pour les utilisateurs)	textBoxModifierf	
Prime (pour les utilisateurs)	textBoxPrimeUti		Prime (pour les utilisateurs)	textBoxModifierf	
Créer employés			Modifié employé		

Tconnexion	Tutilisateur	Tajtmat	TchargeInc	TcreaEmp	Tstat
Formules [Design] x Employees					
Statistiques des utilisateurs					
Primes	listBoxPrime				
☐ Décroissant					
☐ Croissant					
Afficher Primes					
Date d'embauche					
☐ Du plus ancien au plus récent					
☐ Du plus récent au plus ancien					
listBoxDateEmbauche					
Afficher date d'ambauche					

18/11/2025

Remplissage de la base de données création de l’interface sur VSCode et créations de toutes les classes métiers.

Lien pour voir le code: -



Element(s)	Classe
Materiel	CMateriels
Employes	CEmployes
Incident	CIncidents
Utilisateurs	CUtilisateurs
Responsables	CResponsables
Technicien	CTechniciens

Orange : Sanaa
Vert : Adrien

20/11/2025

On va créer une classe BD pour gérer les interactions avec la base de donnée.
Cette classe va devoir:

BLEU : ADRIEN

VERT : SANAA

Insérer:

Des nouveaux employés (fait)

Des nouveaux utilisateurs

//Déjà fait lors de la création d'un employé.

Des nouveaux techniciens

→ Création des if pour les **techniciens(fini)**

Du nouveau matériel (fait)

Des nouveaux incidents (fait)

Supprimer:

Des employés (fait)

Des techniciens(fini)

Des utilisateurs(fini)

Modifier:

Des incidents (fait)

Des employés (fait)

Des techniciens(fini)

Des utilisateurs (fait)

Affiche et récupérer:

Des noms d'employés

Des noms de techniciens

Des noms d'utilisateurs

Les employés et leur prime (décroissant) → pour faire les statistiques (fait)

Des id d'incidents non résolu (fait)

Des id de matériel (fait)

→ n'est pas utile donc je l'ai **supprimé (aussi ce interface)**

Matériel

comboBoxMater ▼

Supprimer Matériel

Infos Matériel

ListBoxInfosMatériel

04/12/2025

Table : utilisateurs

Prime

☐ Décroissant

ListboxPrime

Table : employes

Date

☐ Le + récent

ListboxDateEMB

Ce qu'il y a en ROUGE il ne faut pas le mettre c'est juste

09/12/2025

Gestion de la connexion à notre application en fonction de leur rôle les utilisateurs auront différents onglets disponibles. Par exemple, un utilisateur ne peut que déclarer un incident et suivre son avancement.

A' la connexion de l'application c'est à dire dans le load on a accès qu' au premier onglet.

```
private void Form1_Load(object sender, EventArgs e)
{
    // On a accès que à l'onglet de connexion au début
    for (int i = 1; i < tabControl1.TabPages.Count; i++)
    {
        tabControl1.TabPages[i].Enabled = false;
    }

    // Aller vers le premier onglet
    tabControl1.SelectedIndex = 0;
}
```

Tableau des droit

utilisateur	technicien	responsables
Tutilisateur	TajMat TchargeInc	TcreaEmp Tstat

On va tester la connexion avec trois employés généraux.

Rôle	Matricule	Mdp
responsable	Resp001	12
technicien	Tech002	123
utilisateur	User003	1234

Voici le code de notre bouton de connexion:

```
// Gestion de la connexion à la bd.
private void connexion_Click(object sender, EventArgs e)
{
    // Nettoyage pour passer en utf8 ( comme la BD )
    string matricule = textBoxMatricule.Text.Trim();
    byte[] utf8Bytes = System.Text.Encoding.UTF8.GetBytes(matricule);
    string hexCSharp = BitConverter.ToString(utf8Bytes).Replace("-", "");
    MessageBox.Show($"Matricule saisi : [{matricule}]\nHEX C#: {hexCSharp}");

    string motDePasse = TextBoxMDP.Text.Trim();

    BD maBD = new BD();

    // Connexion
    string role = maBD.Connexion(matricule, motDePasse);

    // Si on n'arrive pas à se connecter
    if (role == null)
    {
        MessageBox.Show("Identifiants incorrects.", "Erreur", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        return;
    }

    // Débogage : afficher le rôle récupéré
    MessageBox.Show($"Connexion réussie.\nRôle récupéré : {role}", "Succès",
    MessageBoxButtons.OK, MessageBoxIcon.Information);

    // Désactiver tous les onglets
    foreach (TabPage tab in tabControl1.TabPages)
        tab.Enabled = false;

    // Activer un onglet selon le rôle
    switch (role)
    {
        case "Utilisateur":
            Tutilisateur.Enabled = true;
            break;

        case "Technicien":
            Tmat.Enabled = true;
            TchargeInc.Enabled = true;
            break;

        case "Responsable":
```



```
        TcreaEmp.Enabled = true;
        Tstat.Enabled = true;
        break;
    }

    // Aller automatiquement vers le premier onglet autorisé
    foreach (TabPage tab in tabControl1.TabPages)
    {
        if (tab.Enabled)
        {
            tabControl1.SelectedTab = tab;
            break;
        }
    }
}
```

Voici le code qui interagit avec la bd:

```
public string Connexion(string matricule, string motDePasse)
{
    // Nettoyage simple du matricule et mot de passe
    matricule = matricule.Trim();
    motDePasse = motDePasse.Trim();

    // Chaîne de connexion avec UTF-8
    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
    SslMode=none; Charset=utf8mb4;";

    try
    {
        using (MySqlConnection conn = new MySqlConnection(connStr))
        {
            conn.Open();

            // Débogage : afficher la base utilisée
            MessageBox.Show("Base utilisée : " + conn.Database, "Info",
            MessageBoxButtons.OK, MessageBoxIcon.Information);

            // Débogage : afficher tous les matricules dans la BD
            string allMatricules = "";
            using (var cmdAll = new MySqlCommand("SELECT Matricule FROM employes",
            conn))
            using (var readerAll = cmdAll.ExecuteReader())
            {
                while (readerAll.Read())
```

```
{
    string mat = readerAll.GetString("Matricule").Trim();
    allMatricules += $"{mat}\n";
}
}
MessageBox.Show("Matricules dans la BD :\n" + allMatricules, "Débogage",
MessageBoxButtons.OK, MessageBoxIcon.Information);

// Requête SQL pour trouver le matricule exact
string query = "SELECT Mot_de_passe, Role FROM employes WHERE
TRIM(Matricule) = @matricule";

using (MySqlCommand cmd = new MySqlCommand(query, conn))
{
    // Ajouter le paramètre après avoir créé le MySqlCommand
    cmd.Parameters.Add("@matricule", MySqlDbType.VarChar, 50).Value =
matricule;

    using (var reader = cmd.ExecuteReader())
    {
        if (!reader.Read())
        {
            MessageBox.Show("Matricule introuvable dans la base de données !",
"Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return null;
        }

        string hashBD = reader.GetString("Mot_de_passe");
        string role = reader.GetString("Role");

        // Débogage : afficher les infos récupérées
        MessageBox.Show($"Matricule trouvé.\nHash dans BD: {hashBD}\nMot de passe
saisi: {motDePasse}\nRôle: {role}",
            "Info", MessageBoxButtons.OK, MessageBoxIcon.Information);

        // Vérification du mot de passe via BCrypt
        if (BCrypt.Net.BCrypt.Verify(motDePasse, hashBD))
            return role;
        else
        {
            MessageBox.Show("Mot de passe incorrect.", "Erreur",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            return null;
        }
    }
}
}
```

```
}  
catch (Exception ex)  
{  
    MessageBox.Show($"Erreur de connexion à la base de données : {ex.Message}",  
"Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);  
    return null;  
}  
}
```

Bouton de déconnexion:

Programme principal

```
// bouton pour se déconnecter  
private void boutonDeconnecter_Click(object sender, EventArgs e)  
{  
  
    // On a accès seulement à l'onglet de connexion  
    for (int i = 1; i < tabControl1.TabPages.Count; i++)  
    {  
        tabControl1.TabPages[i].Enabled = false;  
    }  
  
    // Aller vers le premier onglet  
    tabControl1.SelectedIndex = 0;  
  
    MessageBox.Show("Vous êtes déconnecté");  
  
    // Vider tous les controles  
    ViderControles(tabControl1);  
  
}
```

10/12/2025

Bouton de création d'un matériel

Dans le code principal

```
// Ajouter un matériel  
private void boutonAjouterMat_Click(object sender, EventArgs e)  
{
```

```
BD maBD = new BD();

// récupérer l'id du propriétaire sélectionné
int idProprietaire =
maBD.RecupererIdEmployeParMatricule(ComboBoxPropri.SelectedItem.ToString());

// récupérer la mémoire en int
int memoire = int.Parse(TextboxMemoire.Text);

// Gestion de la garantie
int garantie = 0;

if (radioButtonGarantieOui.Checked)
{
    garantie = 1;
}
else if (radioButtonGarantieNon.Checked)
{
    garantie = 0;
}

// Gestion de l'achat ou location
int achat = 0;

if (radioButtonAchat.Checked)
{
    achat = 1;
}
else if (radioButtonLoué.Checked)
{
    garantie = 0;
}

// On crée le matériel
CMateriels unMateriel = new CMateriels(TextboxProcesseur.Text, memoire,
TextboxDisque.Text, TextboxLogiciels.Text, textBoxFournisseur.Text,
dateTimePickerMat.Value.ToString("yyyy-MM-dd"), garantie, achat, idProprietaire);

// on insère le matériel dans la BD
maBD.InsererMateriel(unMateriel);

// MessageBox de confirmation
MessageBox.Show("Matériel ajouté avec succès !");

// On actualise les combobox
Actualiser();
}
```

Dans la classe Bd:

```
// Insert pour du nouveau matériel
public void InsérerMatériel(CMateriels unMatériel)
{
    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
    SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    MySqlCommand cmd = conn.CreateCommand();
    {
        // On écrit le texte de la commande
        cmd.CommandText = "INSERT INTO materiel(Processeur, Memoire, Disque, Logiciel_,
Fournisseur, Date_achat_ou_loc, Garantie, Acheter, Id_employes) VALUES (@Processeur,
@Memoire, @Disque, @Logiciel_, @Fournisseur, @Date_achat_ou_loc, @Garantie, @Acheter,
@Id_employes)";

        // on récupère ce qu'il nous faut avec la méthode get
        cmd.Parameters.AddWithValue("@Processeur", unMatériel.getProcesseur());
        cmd.Parameters.AddWithValue("@Memoire", unMatériel.getMemoire());
        cmd.Parameters.AddWithValue("@Disque", unMatériel.getDisque());
        cmd.Parameters.AddWithValue("@Logiciel_", unMatériel.getLogiciel());
        cmd.Parameters.AddWithValue("@Fournisseur", unMatériel.getFournisseur());
        cmd.Parameters.AddWithValue("@Date_achat_ou_loc",
unMatériel.getdate_achat_loc());
        cmd.Parameters.AddWithValue("@Garantie", unMatériel.getGarantie());
        cmd.Parameters.AddWithValue("@Acheter", unMatériel.getAchat());
        cmd.Parameters.AddWithValue("@Id_employes", unMatériel.getId_e());

        cmd.ExecuteNonQuery();
    }

    conn.Close();
}
```

Bouton de suppression d'un matériel

Dans le programme principal:

```
private void boutonSupprimerPoste_Click(object sender, EventArgs e)
{
    BD maBD = new BD();
```

```
if (ComboBoxPoste.SelectedItem == null)
{
    MessageBox.Show("Veuillez sélectionner un matériel !");
    return;
}

int idMateriel = Convert.ToInt32(ComboBoxPoste.SelectedItem);

// MessageBox de confirmation
DialogResult result = MessageBox.Show(
    "Êtes-vous sûr de vouloir supprimer ce matériel ?",
    "Confirmation",
    MessageBoxButtons.YesNo,
    MessageBoxIcon.Warning
);

if (result == DialogResult.Yes)
{
    maBD.SupprimerMateriel(idMateriel);
    MessageBox.Show("Le matériel a bien été supprimé.");

    comboBoxMateriel.Items.Clear();
    Actualiser();
}

else
{
    MessageBox.Show("Suppression annulée.");
}
}

Dans la classe bd
// Cette fonction va supprimer un matériel de la base de donnée
public void SupprimerMateriel(int idMateriel)
{
    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=; SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    MySqlCommand cmd = conn.CreateCommand();
    {
        // On écrit le texte de la commande
```

```
cmd.CommandText = "DELETE FROM materiel WHERE Id_materiel =  
@Id_materiel";  
  
// on récupère ce qu'il nous faut.  
cmd.Parameters.AddWithValue("@Id_materiel", idMateriel);  
cmd.ExecuteNonQuery();  
}  
conn.Close();  
}
```

Bouton de description d'un matériel

Dans le programme principal

```
// Cette fonction va afficher les infos du matériel sélectionné  
private void buttonInfosMateriel_Click(object sender, EventArgs e)  
{  
  
    BD maBD = new BD();  
    int idMateriel = Convert.ToInt32(ComboBoxPoste.SelectedItem);  
  
    // On récupère les infos du matériel  
    string infos = maBD.InfosMateriel(idMateriel);  
  
    // On affiche les infos dans la listbox  
    string[] lignes = infos.Split(new[] { '\n' }, StringSplitOptions.RemoveEmptyEntries);  
    listBoxInfosMat.Items.Clear(); // vider avant d'ajouter  
    foreach (string ligne in lignes)  
    {  
        listBoxInfosMat.Items.Add(ligne);  
    }  
}
```

Dans la classe bd.

```
// Cette fonction va récupérer les informations d'un matériel à partir de son id  
public string InfosMateriel(int idMateriel)  
{  
  
    // une méthode de connexion à la base de donnée projet_bd  
    MySqlConnection conn;  
  
    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;  
SslMode=none;";  
    conn = new MySqlConnection(connStr);  
  
    conn.Open();
```

```
// On crée la requête pour récupérer les informations du matériel
string requete = "SELECT * FROM materiel WHERE Id_materiel = @Id_materiel";
MySQLCommand cmd = new MySQLCommand(requete, conn);
cmd.Parameters.AddWithValue("@Id_materiel", idMateriel);

// On crée une liste pour stocker les informations du matériel
using (MySQLDataReader reader = cmd.ExecuteReader())
{
    if (reader.Read())
    {
        string infos = $"--- Informations sur le matériel ---\n" +
            $"Processeur : {reader["Processeur"]}\n" +
            $"Mémoire : {reader["Memoire"]}\n" +
            $"Disque : {reader["Disque"]}\n" +
            $"Logiciels : {reader["Logiciel"]}\n" +
            $"Fournisseur : {reader["Fournisseur"]}\n" +
            $"Date d'achat/location :
{Convert.ToDateTime(reader["Date_achat_ou_loc"]):yyyy-MM-dd}\n" +
            $"Garantie : {(bool)reader["Garantie"] ? "Oui" : "Non"}\n" +
            $"Achat : {(bool)reader["Acheter"] ? "Oui" : "Non"}\n" +
            $"ID Employé : {reader["Id_employes"]}";

        // Retourne les informations du matériel
        return infos;
    }
}
// Si il n'y a pas de matériel avec cet id on retourne un message d'erreur
return "Aucun matériel trouvé pour cet ID.";
}
```

Bouton de déclaration d'un incident.

Programme principal:

```
// Un utilisateur déclare un incident sur un poste
private void buttonDeclarerIncident_Click(object sender, EventArgs e)
{
    BD maBD = new BD();

    // Récupérer l'id du propriétaire sélectionné
    int idProprietaire = CSessionUtilisateur.IdEmploye;

    // Récupérer l'id du poste concerné
    int idPoste = Convert.ToInt32(ComboBoxPoste.SelectedItem);

    // Date de déclaration de l'incident
```



```
DateTime maintenant = DateTime.Now;

// Date de l'incident
DateTime dateIncident = maintenant.Date; // juste la date (00:00:00)

// Heure de l'incident
DateTime heureDebut = maintenant;

// Heure de résolution de l'incident (null au début)
DateTime? heureFin = null;

// On crée l'incident
CIncidents unIncident = new CIncidents("EN ATTENTE", dateIncident, heureDebut,
heureFin, TextBoxDesc.Text, "Aucune", 0, idProprietaire, idPoste);

// on insère l'incident dans la BD
maBD.InsererIncident(unIncident);

MessageBox.Show($"Incident déclaré patienté avant sa prise en charge");

// On actualise les combobox
Actualiser();
}

Dans la classe BD:
// Insert pour un nouvel incident
public void InsererIncident(CIncidents unIncidents)
{
    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    MySqlCommand cmd = conn.CreateCommand();
    {
        // On écrit le texte de la commande
        cmd.CommandText = "INSERT INTO incident(Etat, Date_, HeureDeb, HeureFin, Descr,
Type_priseen_charge, id_technicien, id_utilisateur, id_materiel) VALUES (@Etat, @Date_,
@HeureDeb, @HeureFin, @Descr, @Type_priseen_charge, @id_technicien, @id_utilisateur,
@id_materiel)";

        // on récupère ce qu'il nous faut avec la méthode get
        cmd.Parameters.AddWithValue("@Etat", unIncidents.getEtat());
    }
}
```

```
cmd.Parameters.AddWithValue("@Date_", unIncidents.getDate());
cmd.Parameters.AddWithValue("@HeureDeb", unIncidents.getHeureDeb());
cmd.Parameters.AddWithValue("@HeureFin", unIncidents.getHeureFin());
cmd.Parameters.AddWithValue("@Descr", unIncidents.getDescr());
cmd.Parameters.AddWithValue("@Type_priseen_charge",
unIncidents.getTypeTravail());
cmd.Parameters.AddWithValue("@id_techicien", unIncidents.getId_tech());
cmd.Parameters.AddWithValue("@id_utilisateur", unIncidents.getId_uti());
cmd.Parameters.AddWithValue("@id_materiel", unIncidents.getId_mat());
cmd.ExecuteNonQuery();
}

conn.Close();
}
```

11/12/2025

Bouton de suivi d'un incident

Dans le programme principal:

```
// ce bouton va permettre à un utilisateur de suivre l'avancement des incidents qu'il a
déclaré
private void BtAvancement_Click(object sender, EventArgs e)
{
    BD maBD = new BD();

    // Récupérer l'id du propriétaire sélectionné
    int idProprietaire = CSessionUtilisateur.IdEmploye;

    // On appelle la fonction pour récupérer les infos des incidents de l'utilisateur
    string infos = maBD.SuiviIncidentParIdUtilisateur(idProprietaire);

    // On affiche les infos dans la listBox
    string[] lignes = infos.Split(new[] { '\n' }, StringSplitOptions.RemoveEmptyEntries);
    listBoxInfosMat.Items.Clear(); // vider avant d'ajouter
    foreach (string ligne in lignes)
    {
        ListBoxAvancement.Items.Add(ligne);
    }
}
```

Dans la classe BD:

```
// Cette fonction va suivre l'avancement de l'incident selon l'id de la personne qui l'a créé
public string SuiviIncidentParIdUtilisateur(int idUtilisateur)
{
    // une méthode de connexion à la base de donnée projet_bd
```

```
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    // On crée la requête pour récupérer les informations de l'incident
    string requete = "SELECT Etat, Type_priseen_charge, HeureDeb, HeureFin FROM
incident WHERE Id_utilisateur = @idUtilisateur";
    MySqlCommand cmd = new MySqlCommand(requete, conn);
    cmd.Parameters.AddWithValue("@idUtilisateur", idUtilisateur);

    // On crée une liste pour stocker les informations de l'incident
    StringBuilder infos = new StringBuilder();
    using (MySqlDataReader reader = cmd.ExecuteReader())
    {
        int compteur = 1;

        while (reader.Read())
        {
            infos.AppendLine($"--- Incident n°{compteur} ---");
            infos.AppendLine($"Etat de l'incident : {reader["Etat"]}");
            infos.AppendLine($"Type de prise en charge : {reader["Type_priseen_charge"]}");
            infos.AppendLine($"Incident déclaré à : {reader["HeureDeb"]}");
            infos.AppendLine($"Incident résolu à : {reader["HeureFin"]}");
            infos.AppendLine("");

            compteur++;
        }

        conn.Close();

        if (infos.Length == 0)
            return "Aucun incident trouvé pour cet utilisateur.";

        return infos.ToString();
    }
}
```

Bouton de prise en charge d'un incident

Dans le programme principal:

```
// Les techniciens prennent en charge un incident
private void boutonPrendreEnCharge_Click(object sender, EventArgs e)
```

```
{  
    BD maBD = new BD();  
  
    // Récupérer l'id du technicien qui va résoudre l'incident  
    int idTechnicien = CSessionUtilisateur.IdEmploye;  
  
    // Récupérer l'id de l'incident sélectionné  
    int idIncident = Convert.ToInt32(comboBoxIncidentsNonPrisEnCharge.SelectedItem);  
  
    maBD.ModifierIncidentPrendreEncharge(idIncident, idTechnicien,  
richTextBoxTypePriseEnCharge.Text);  
  
    // MessageBox de confirmation  
    DialogResult result = MessageBox.Show("L'incident a bien été pris en charge."  
);  
  
    // On actualise les combobox  
    Actualiser();  
}
```

Dans la classe bd:

```
// Cette fonction va modifier un incident quand il est pris en charge par un technicien  
public void ModifierIncidentPrendreEncharge(int Id_incident, int Id_technicien, string  
typePriseEncharge)  
{  
    // une méthode de connexion à la base de donnée projet_bd  
    MySqlConnection conn;  
  
    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;  
SslMode=none;";  
    conn = new MySqlConnection(connStr);  
  
    conn.Open();  
  
    MySqlCommand cmd = conn.CreateCommand();  
    {  
        // On écrit le texte de la commande  
        cmd.CommandText = "UPDATE incident SET Id_technicien = @Id_technicien, Etat =  
@Etat, Type_priseen_charge = @typePriseEncharge WHERE Id_incident = @Id_incident";  
  
        cmd.Parameters.AddWithValue("@Id_technicien", Id_technicien);  
        cmd.Parameters.AddWithValue("@Etat", "En cours");  
        cmd.Parameters.AddWithValue("@typePriseEncharge", typePriseEncharge);  
        cmd.Parameters.AddWithValue("@Id_incident", Id_incident);  
    }  
}
```

```
        cmd.ExecuteNonQuery();
    }
    conn.Close();
}
```

Bouton de résolution d'un incident

Dans le programme principal:

// Cette fonction va permettre au technicien de résoudre un incident et de décrire le travail effectué

```
private void buttonResoudre_Click(object sender, EventArgs e)
{
    BD maBD = new BD();

    // Date de résolution de l'incident
    DateTime maintenant = DateTime.Now;

    // Heure de la résolution de l'incident
    DateTime heureFin = maintenant;

    // Récupérer l'id de l'incident sélectionné
    int idIncident = Convert.ToInt32(comboBoxResoudre.SelectedItem);

    maBD.ModifierIncidentResoudre(idIncident, richTextBoxDescriptionTravail.Text,
    heureFin);

    // MessageBox de confirmation
    DialogResult result = MessageBox.Show("Incident résolu avec succès."
    );

    // On actualise les combobox
    Actualiser();
}
```

Dans le classe BD:

```
// Cette fonction va modifier un incident quand il est terminé par un technicien
public void ModifierIncidentResoudre(int Id_incident, string descriptionTravail, DateTime
HeureFin)
{
    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);
```

```
conn.Open();

MySQLCommand cmd = conn.CreateCommand();
{
    // On écrit le texte de la commande
    cmd.CommandText = "UPDATE incident SET Etat = @Etat, Descr =
@descriptionTravail, HeureFin = @HeureFin WHERE Id_incident = @Id_incident";

    // on récupère ce qu'il nous f
    cmd.Parameters.AddWithValue("@Etat", "Résolu");
    cmd.Parameters.AddWithValue("@descriptionTravail", descriptionTravail);
    cmd.Parameters.AddWithValue("@Id_incident", Id_incident);
    cmd.Parameters.AddWithValue("@HeureFin", HeureFin);
    cmd.ExecuteNonQuery();
}
conn.Close();
}
```

Bouton de création d'un employé

Dans le programme principal.

// Les responsables crée un nouvel employés

```
private void buttonCreerEmployes_Click(object sender, EventArgs e)
```

```
{
    // on gère le rôle
    string role = "";

    BD maBD = new BD();

    int niveau = 0;

    // si l'employés est un technicien.
    if (radioButtonTechnicien.Checked)
    {
        role = "Technicien";

        if (radioBac.Checked) niveau = 1;
        else if (radioBTS.Checked) niveau = 2;
        else if (radioMaster.Checked) niveau = 3;
    }

    // si l'employés est un utilisateur
    else if (radioButtonUtilisateur.Checked)
    {
        role = "Utilisateur";
    }
}
```

```
// On désactive les radios des techniciens
radioBac.Enabled = false;
radioBTS.Enabled = false;
radioMaster.Enabled = false;
}

// On crée l'employé
CEmployes unEmployes = new CEmployes(textBoxMatricule.Text, textBoxNom.Text,
textBoxPrenom.Text, dateTimePickerE.Value.ToString("yyyy-MM-dd"), textBoxRegion.Text,
txtBoxMDPE.Text, role);

// Récupère l'ID de l'employé créé
int idEmployeCree = maBD.InsererEmployer(unEmployes);

// Ajouter les techniciens si besoin
if (radioButtonTechnicien.Checked)
{
    CTechniciens unTechniciens = new CTechniciens(niveau, idEmployeCree);
    maBD.InsertTechniciens(unTechniciens, idEmployeCree);
}

// Ajouter les utilisateurs si besoin
else if (radioButtonUtilisateur.Checked)
{
    string objectif = textBoxObjectifsUtilisateurs.Text;
    float prime = float.Parse(textBoxPrimeUtilisateur.Text);
    float budget = float.Parse(textBoxPrimeUtilisateur.Text);

    CUtilisateurs unUtilisateurs = new CUtilisateurs(objectif, prime, idEmployeCree);

    maBD.InsererUtilisateur(unUtilisateurs, idEmployeCree);
}

// MessageBox de confirmation
DialogResult result = MessageBox.Show("Un nouvel employé a été créé"
);

// Actualiser
Actualiser();
}
```

Dans la classe Bd.

```
// Cette requête va insérer un nouvel employé dans la base de donnée
public int InsererEmployer(CEmployes unEmploye)
{
}
```

```
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    string MdpSecurise = BCrypt.Net.BCrypt.HashPassword(unEmploye.getMdpE());

    MySqlCommand cmd = conn.CreateCommand();
    {
        cmd.CommandText = "INSERT INTO employes(Matricule, Nom, Prenom,
Date_Embauche, Regions, Mot_de_passe, Role) " +
            "VALUES (@Matricule, @Nom, @Prenom, @Date_embauche, @Regions,
@Mot_de_passe, @Role); " +
            "SELECT LAST_INSERT_ID();";

        cmd.Parameters.AddWithValue("@Matricule", unEmploye.getMatriculeE());
        cmd.Parameters.AddWithValue("@Nom", unEmploye.getNomE());
        cmd.Parameters.AddWithValue("@Prenom", unEmploye.getPrenomE());
        cmd.Parameters.AddWithValue("@Date_embauche", unEmploye.getDateEmbaucheE());
        cmd.Parameters.AddWithValue("@Regions", unEmploye.getRegionsE());
        cmd.Parameters.AddWithValue("@Mot_de_passe", MdpSecurise);
        cmd.Parameters.AddWithValue("@Role", unEmploye.getRoleE());

        // Récupération de l'ID généré
        int id = Convert.ToInt32(cmd.ExecuteScalar());

        conn.Close();

        return id;
    }
}

// Insert pour un nouveau technicien
public void InsertTechniciens(CTechniciens unTechniciens, int idEmploye)
{
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    MySqlCommand cmd = conn.CreateCommand();
    {
```



```
cmd.CommandText = "INSERT INTO technicien (Niveau_formation, Id_employes) " +
    "VALUES (@Niveau, @id_employes)";

cmd.Parameters.AddWithValue("@Niveau", unTechniciens.getNivFormation());
cmd.Parameters.AddWithValue("@id_employes", idEmploye);

cmd.ExecuteNonQuery();
}

conn.Close();
}

//Insert pour un nouvel utilisateur
public void InsérerUtilisateur(CUtilisateurs unUtilisateurs, int idEmploye)
{
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    MySqlCommand cmd = conn.CreateCommand();
    {
        cmd.CommandText = "INSERT INTO utilisateurs (Objectif, Prime, Id_employes) " +
            "VALUES (@Objectif, @Prime, @Id_employes)";

        cmd.Parameters.AddWithValue("@Objectif", unUtilisateurs.getUtilisateurObjectif());
        cmd.Parameters.AddWithValue("@Prime", unUtilisateurs.getUtilisateurPrime());
        cmd.Parameters.AddWithValue("@Id_employes", idEmploye);

        cmd.ExecuteNonQuery();
    }

    conn.Close();
}
```

Bouton de suppression d'un employé

Programme principal

```
private void buttonSupprimerEmployes_Click(object sender, EventArgs e)
{
    BD maBD = new BD();

    // On vérifie quel rôle est sélectionné et on supprime l'employé correspondant
    if (comboBoxModifierEmployesTechnicien.SelectedItem != null)
```

```
{
    string role = "Technicien";

    // d'abord on récupère l'id du technicien à supprimer
    string matricule = comboBoxModifierEmployesTechnicien.SelectedItem.ToString();
    int idTechnicien = maBD.RecupererIdEmployeParMatricule(matricule);

    // MessageBox de confirmation
    DialogResult result = MessageBox.Show(
        "Êtes-vous sûr de vouloir supprimer ce technicien ?",
        "Confirmation",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Warning
    );

    if (result == DialogResult.Yes)
    {
        maBD.SupprimerTechnicien(idTechnicien);
        MessageBox.Show("Le technicien a bien été supprimé.");
    }
}

else if (comboBoxModifierEmployesUtilisateur.SelectedItem != null)
{
    string role = "Utilisateur";

    // d'abord on récupère l'id de l'utilisateur à supprimer
    string matricule = comboBoxModifierEmployesUtilisateur.SelectedItem.ToString();
    int idUtilisateur = maBD.RecupererIdEmployeParMatricule(matricule);

    // MessageBox de confirmation
    DialogResult result = MessageBox.Show(
        "Êtes-vous sûr de vouloir supprimer cet utilisateur ?",
        "Confirmation",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Warning
    );

    if (result == DialogResult.Yes)
    {
        maBD.SupprimerUtilisateur(idUtilisateur);
        MessageBox.Show("L'utilisateur a bien été supprimé.");
    }
}

// On vide les sélections des combobox
comboBoxModifierEmployesTechnicien.SelectedItem = null;
comboBoxModifierEmployesUtilisateur.SelectedItem = null;
```

```
comboBoxModifierEmployesTechnicien.Items.Clear();  
comboBoxModifierEmployesUtilisateur.Items.Clear();
```

```
// On actualise les combobox  
Actualiser();  
}
```

Dans la classe Bd

```
// Cette fonction va supprimer un employé de la base de donnée  
public void SupprimerEmploye(string Matricule)  
{  
    // une méthode de connexion à la base de donnée projet_bd  
    MySqlConnection conn;  
  
    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;  
    SslMode=none;";  
    conn = new MySqlConnection(connStr);  
  
    conn.Open();  
  
    MySqlCommand cmd = conn.CreateCommand();  
    {  
        // On écrit le texte de la commande  
        cmd.CommandText = "DELETE FROM employes WHERE Matricule = @Matricule";  
  
        // on récupère ce qu'il nous faut avec la méthode get ( ici l'id de l'employé )  
        cmd.Parameters.AddWithValue("@Matricule", Matricule);  
        cmd.ExecuteNonQuery();  
    }  
    conn.Close();  
}
```

```
// Cette fonction va supprimer un utilisateur de la base de donnée  
public void SupprimerUtilisateur(int IdEmployes)  
{  
    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;  
    SslMode=none;";  
    MySqlConnection conn = new MySqlConnection(connStr);  
  
    try  
    {  
        conn.Open();  
        MySqlTransaction transaction = conn.BeginTransaction();  
  
        try  
        {  
            // D'abord on supprime l'utilisateur de la table utilisateurs  
            MySqlCommand cmd = new MySqlCommand();
```

```
        cmd.Connection = conn;
        cmd.Transaction = transaction;
        cmd.CommandText = "DELETE FROM utilisateurs WHERE Id_employes =
@Id_employes";
        cmd.Parameters.AddWithValue("@Id_employes", IdEmployes);
        cmd.ExecuteNonQuery();
        cmd.Parameters.Clear();

        // puis l'employé de la table employes
        MySqlCommand cmd2 = new MySqlCommand();
        cmd2.Connection = conn;
        cmd2.Transaction = transaction;
        cmd2.CommandText = "DELETE FROM employes WHERE Id_employes =
@Id_employes";
        cmd2.Parameters.AddWithValue("@Id_employes", IdEmployes);
        cmd2.ExecuteNonQuery();
        cmd2.Parameters.Clear();

        transaction.Commit();
    }
    catch (Exception ex)
    {
        transaction.Rollback();
        MessageBox.Show("Erreur lors de la suppression : " + ex.Message);
    }
}
catch (Exception ex)
{
    MessageBox.Show("Erreur de connexion à la base : " + ex.Message);
}
finally
{
    if (conn.State == System.Data.ConnectionState.Open)
        conn.Close();
}
}
```

```
// Cette fonction va supprimer un technicien de la base de donnée
public void SupprimerTechnicien(int IdEmployes)
{
    // D'abord on supprime le technicien de la table technicien
    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
```

```
conn = new MySqlConnection(connStr);

conn.Open();

MySQLCommand cmd = conn.CreateCommand();
{
    // On écrit le texte de la commande
    cmd.CommandText = "DELETE FROM technicien WHERE Id_employes =
@Id_employes";

    // on supprime ce qu'il nous faut avec la méthode get ( ici l'id de l'employé )
    cmd.Parameters.AddWithValue("@Id_employes", IdEmployes);
    cmd.ExecuteNonQuery();
}

// Ensuite on supprime l'employé de la table employes
MySQLCommand cmd2 = conn.CreateCommand();
{
    // On écrit le texte de la commande
    cmd2.CommandText = "DELETE FROM employes WHERE Id_employes =
@Id_employes";

    // on supprime ce qu'il nous faut avec la méthode get ( ici l'id de l'employé )
    cmd2.Parameters.AddWithValue("@Id_employes", IdEmployes);
    cmd2.ExecuteNonQuery();
}

conn.Close();
}
```

Bouton de modification d'un employé

Dans le code principal

```
// Ce bouton va modifier un employé
private void buttonModifierEmploye_Click(object sender, EventArgs e)
{
    BD maBD = new BD();

    // On vérifie quel rôle est sélectionné et on modifie l'employé correspondant
    if (comboBoxModifierEmployesTechnicien.SelectedItem != null)
    {
        // d'abord on récupère l'id du technicien à modifier
        string matricule = comboBoxModifierEmployesTechnicien.SelectedItem.ToString();
        int idTechnicien = maBD.RecupererIdEmployeParMatricule(matricule);
    }
}
```

```
String roleE = "Technicien";
int niveau = 0;

if (radioButtonModifierBac.Checked)
    niveau = 1;
else if (radioButtonModifierBTS.Checked)
    niveau = 2;
else if (radioButtonModifierMaster.Checked)
    niveau = 3;

// On crée l'employé
CEmployes unEmployes = new CEmployes(idTechnicien, textBoxModifierMatricule.Text,
textBoxModifierNom.Text,
textBoxModifierPrenom.Text,dateTimePickerModifierDateEmbauche.Value.ToString("yyyy-
MM-dd"),textBoxRegion.Text,roleE);

// On crée le technicien
CTechniciens unTechniciens = new CTechniciens(niveau, idTechnicien);

// puis on modifie l'employé dans la bd
maBD.ModifierEmploye(unEmployes);
maBD.ModifierTechnicien(unTechniciens, idTechnicien);
}

else if (comboBoxModifierEmployesUtilisateur.SelectedItem != null)
{
    // d'abord on récupère l'id de l'utilisateur à modifier
    string matricule = comboBoxModifierEmployesUtilisateur.SelectedItem.ToString();
    int idUtilisateur = maBD.RecupererIdEmployeParMatricule(matricule);

    String roleE = "Utilisateur";

    // On crée l'employé
    CEmployes unEmployes = new CEmployes(idUtilisateur, textBoxModifierMatricule.Text,
textBoxModifierNom.Text, textBoxModifierPrenom.Text,
dateTimePickerModifierDateEmbauche.Value.ToString("yyyy-MM-dd"), textBoxRegion.Text,
roleE);

    // On crée l'utilisateur
    CUtilisateurs unUtilisateurs = new CUtilisateurs(textBoxModifierObjectif.Text,
float.Parse(textBoxModifierPrime.Text), idUtilisateur);

    // puis on modifie l'employé dans la bd
    maBD.ModifierEmploye(unEmployes);
    maBD.ModifierUtilisateur(unUtilisateurs, idUtilisateur);
}
```

```
// On vide les sélections des combobox
comboBoxModifierEmployesTechnicien.SelectedItem = null;
comboBoxModifierEmployesUtilisateur.SelectedItem = null;
comboBoxModifierEmployesTechnicien.Items.Clear();
comboBoxModifierEmployesUtilisateur.Items.Clear();

// MessageBox de confirmation
DialogResult result = MessageBox.Show("L'employé a bien été modifié.");

// On actualise les combobox
Actualiser();
}

Dans la classe Bd:

// Cette fonction va modifier un employés
public bool ModifierEmploye(CEmployes unEmploye)
{
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    try
    {
        conn.Open();

        MySqlCommand cmd = conn.CreateCommand();
        cmd.CommandText = @"UPDATE employes
SET Matricule = @Matricule,
    Nom = @Nom,
    Prenom = @Prenom,
    Date_Embauche = @Date_embauche,
    Regions = @Regions,
    Role = @Role
WHERE Id_employes = @Id_employes";

        // on récupère ce qu'il nous faut avec la méthode get
        cmd.Parameters.AddWithValue("@Id_employes", unEmploye.getIdEmploye());
        cmd.Parameters.AddWithValue("@Matricule", unEmploye.getMatriculeE());
        cmd.Parameters.AddWithValue("@Nom", unEmploye.getNomE());
        cmd.Parameters.AddWithValue("@Prenom", unEmploye.getPrenomE());
        cmd.Parameters.AddWithValue("@Date_embauche",
unEmploye.getDateEmbaucheE());
        cmd.Parameters.AddWithValue("@Regions", unEmploye.getRegionsE());
        cmd.Parameters.AddWithValue("@Role", unEmploye.getRoleE());
```

```
        int lignesAffectees = cmd.ExecuteNonQuery();
        conn.Close();

        return lignesAffectees > 0;
    }

    // Gestion des exceptions
    catch (Exception ex)
    {
        MessageBox.Show("Erreur lors de la modification : " + ex.Message);
        if (conn.State == System.Data.ConnectionState.Open)
            conn.Close();
        return false;
    }
}

// Cette fonction va modifier les informations d'un utilisateur
public void ModifierUtilisateur(CUtilisateurs unUtilisateurs, int idEmploye)
{
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    MySqlCommand cmd = conn.CreateCommand();
    {
        cmd.CommandText = @"UPDATE utilisateurs
SET Objectif = @Objectif,
    Prime = @Prime
WHERE Id_employes = @Id_employes";

        cmd.Parameters.AddWithValue("@Objectif",
unUtilisateurs.getUtilisateurObjectif());
        cmd.Parameters.AddWithValue("@Prime", unUtilisateurs.getUtilisateurPrime());
        cmd.Parameters.AddWithValue("@Id_employes", idEmploye);

        cmd.ExecuteNonQuery();
    }

    conn.Close();
}
```



```
// Cette fonction va modifier les informations d'un technicien
public void ModifierTechnicien(CTechniciens unTechniciens, int idEmploye)
{
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    MySqlCommand cmd = conn.CreateCommand();
    {
        cmd.CommandText = @"UPDATE technicien
SET Niveau_formation = @Niveau_formation
WHERE Id_employes = @Id_employes";

        cmd.Parameters.AddWithValue("@Niveau_formation",
unTechniciens.getNivFormation());
        cmd.Parameters.AddWithValue("@Id_employes", idEmploye);

        cmd.ExecuteNonQuery();
    }

    conn.Close();
}
```

Bouton de statistiques de primes

Dans le code principal

```
// Les responsables affichent les statistiques des primes des utilisateurs
private void boutonAfficherStatistiquesPrime_Click(object sender, EventArgs e)
{
    BD maBD = new BD();
    if (radioButtonCroissant.Checked)
    {
        // créer une liste pour stocker les résultats
        List<string> resultats = maBD.ClasserUtilisateursParPrimeCroissant();

        // Vider la ListBox avant d'ajouter
        listBoxPrime.Items.Clear();

        foreach (string ligne in resultats)
        {
```

```
        listBoxPrime.Items.Add(ligne);
    }
}

else if (radioButtonDecroissant.Checked)
{
    // créer une liste pour stocker les résultats
    List<string> resultats = maBD.ClasserUtilisateursParPrimeDecroissant();

    // Vider la ListBox avant d'ajouter
    listBoxPrime.Items.Clear();

    foreach (string ligne in resultats)
    {
        listBoxPrime.Items.Add(ligne);
    }
}
}
```

Dans la classe Bd:

```
// Cette fonction va créer une liste qui va classer les utilisateurs par prime ( décroissant )
( utile pour les statistiques )
public List<string> ClasserUtilisateursParPrimeDecroissant()
{
    // On crée une liste pour stocker les informations des utilisateurs
    List<string> listeUtilisateurs = new List<string>();

    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    string requete = "SELECT e.Nom, e.Prenom, u.Prime FROM employes e, utilisateurs u
WHERE e.Id_employes = u.Id_employes\r\nORDER BY u.Prime DESC;";
    MySqlCommand cmd = new MySqlCommand(requete, conn);

    // Exécution de la commande
    using (MySqlDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            // on ajoute les informations pour la liste
            string nom = reader.GetString("Nom");
```

```
        string prenom = reader.GetString("Prenom");
        decimal prime = reader.GetDecimal("Prime");

        listeUtilisateurs.Add($"{nom} {prenom} a une prime de {prime}€");
    }
}

conn.Close();

return listeUtilisateurs;
}
// Cette fonction va créer une liste qui va classer les utilisateurs par prime ( croissant )
( utile pour les statistiques )
public List<string> ClasserUtilisateursParPrimeCroissant()
{
    // On crée une liste pour stocker les informations des utilisateurs
    List<string> listeUtilisateurs = new List<string>();

    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    string requete = "SELECT e.Nom, e.Prenom, u.Prime\r\nFROM employes e, utilisateurs
u\r\nWHERE e.Id_employes = u.Id_employes\r\nORDER BY u.Prime ASC;";
    MySqlCommand cmd = new MySqlCommand(requete, conn);

    // Exécution de la commande
    using (MySqlDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            // on ajoute les informations pour la liste
            string nom = reader.GetString("Nom");
            string prenom = reader.GetString("Prenom");
            decimal prime = reader.GetDecimal("Prime");

            listeUtilisateurs.Add($"{nom} {prenom} a une prime de {prime}€");
        }
    }

    conn.Close();

    return listeUtilisateurs;
}
```

```
}
```

Bouton de statistiques de date d'embauche

Dans le programme principal

```
private void buttonAfficherDateEmbauche_Click(object sender, EventArgs e)
{
    // les responsables affichent les statistiques des dates d'embauche des employés
    BD maBD = new BD();

    // si on choisit du plus ancien au plus récent
    if (radioButtonDateEmbaucheCroissant.Checked)
    {
        // créer une liste pour stocker les résultats
        List<string> resultats = maBD.ClasserEmployesParDateCroissant();

        // Vider la ListBox avant d'ajouter
        listBoxDateEmbauche.Items.Clear();

        // lire la liste et ajouter chaque ligne à la ListBox
        foreach (string ligne in resultats)

        {
            listBoxDateEmbauche.Items.Add(ligne);
        }
    }

    // si on choisit du plus récent au plus ancien
    else if (radioButtonDateEmbaucheDecroissant.Checked)
    {
        // créer une liste pour stocker les résultats
        List<string> resultats = maBD.ClasserEmployesParDateDecroissant();
        // Vider la ListBox avant d'ajouter
        listBoxDateEmbauche.Items.Clear();
        // lire la liste et ajouter chaque ligne à la ListBox
        foreach (string ligne in resultats)
        {
            listBoxDateEmbauche.Items.Add(ligne);
        }
    }
}
```

Dans la classe Bd:

// Cette fonction va créer une liste qui va classer les employés par date d'embauche (du plus ancien au plus récent)

```
public List<string> ClasserEmployesParDateCroissant()  
{
```

```
// On crée une liste pour stocker les informations des utilisateurs  
List<string> listeEmployes = new List<string>();
```

```
// une méthode de connexion à la base de donnée projet_bd  
MySqlConnection conn;
```

```
string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;  
SslMode=none;";
```

```
conn = new MySqlConnection(connStr);
```

```
conn.Open();
```

```
string requete = "SELECT e.Nom, e.Prenom, e.Date_embauche FROM employes e  
ORDER BY e.Date_embauche ASC;";
```

```
MySqlCommand cmd = new MySqlCommand(requete, conn);
```

```
// Exécution de la commande
```

```
using (MySqlDataReader reader = cmd.ExecuteReader())
```

```
{
```

```
while (reader.Read())
```

```
{
```

```
// on ajoute les informations pour la liste
```

```
string nom = reader.GetString("Nom");
```

```
string prenom = reader.GetString("Prenom");
```

```
DateTime date = reader.GetDateTime("Date_embauche");
```

```
listeEmployes.Add($"{nom} {prenom} a été embauché le {date}");
```

```
}
```

```
}
```

```
conn.Close();
```

```
// retour de notre liste d'employés
```

```
return listeEmployes;
```

```
}
```

// Cette fonction va créer une liste qui va classer les employés par date d'embauche (du plus récent au plus ancien)

```
public List<string> ClasserEmployesParDateDecroissant()  
{
```

```
// On crée une liste pour stocker les informations des utilisateurs
```

```
List<string> listeEmployes = new List<string>();
```

```
// une méthode de connexion à la base de donnée projet_bd
```

```
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    string requete = "SELECT e.Nom, e.Prenom, e.Date_embauche FROM employes e
ORDER BY e.Date_embauche DESC;";
    MySqlCommand cmd = new MySqlCommand(requete, conn);

    // Exécution de la commande
    using (MySqlDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            // on ajoute les informations pour la liste
            string nom = reader.GetString("Nom");
            string prenom = reader.GetString("Prenom");
            DateTime date = reader.GetDateTime("Date_embauche");

            listeEmployes.Add($"{nom} {prenom} a été embauché le {date}");
        }
    }

    conn.Close();

    // retour de notre liste d'employés
    return listeEmployes;
}
```

Bouton de statistiques de résolution d'incident

Dans le programme principal

```
// Ce bouton affiche le temps moyen de résolution des incidents
private void boutonTempsMoyenDeResolution_Click(object sender, EventArgs e)
{
    BD maBD = new BD();

    // créer une liste pour stocker les résultats
    List<string> resultats = maBD.TempsMoyenResolutionParTechnicien();

    // Vider la ListBox avant d'ajouter
```

```
        listBoxTempsMoyenRésolution.Items.Clear();

        // lire la liste et ajouter chaque ligne à la ListBox
        foreach (string ligne in resultats)

        {
            listBoxTempsMoyenRésolution.Items.Add(ligne);
        }
    }
}
```

Dans la classe Bd

// Cette fonction va créer une liste qui va afficher le temps moyen de résolution des incidents par technicien

```
public List<string> TempsMoyenResolutionParTechnicien()
{
    List<string> listeTechniciens = new List<string>();

    // une méthode de connexion à la base de donnée projet_bd
    MySqlConnection conn;

    string connStr = "Server=127.0.0.1; Database=projet_bd; Uid=root; Password=;
    SslMode=none;";
    conn = new MySqlConnection(connStr);

    conn.Open();

    string requete =
        "SELECT e.Nom, e.Prenom, " +
        "SEC_TO_TIME(AVG(TIMESTAMPDIFF(SECOND, i.HeureDeb, i.HeureFin))) AS
TempsMoyen " +
        "FROM incident i, technicien t, employes e " +
        "WHERE e.Id_employes = t.Id_employes " +
        "AND t.Id_employes = i.Id_technicien " +
        "AND i.Etat = 'Résolu' " +
        "AND i.HeureDeb IS NOT NULL " +
        "AND i.HeureFin IS NOT NULL " +
        "GROUP BY t.Id_technicien;";

    MySqlCommand cmd = new MySqlCommand(requete, conn);

    // Exécution de la commande
    using (MySqlDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            // on ajoute les informations pour la liste

```

```
    string nom = reader.GetString("Nom");
    string prenom = reader.GetString("Prenom");
    string tempsMoyen = reader.GetString("TempsMoyen");

    listeTechniciens.Add($"{nom} {prenom} a un temps moyen de résolution de
{tempsMoyen}");
    }
}

conn.Close();

return listeTechniciens;
}
```

13/12/2025

Création du .exe et du guide d'utilisation